# Deriving Microscopic RTOS Constraints from Macroscopic Drone Physics

**Objective:** To mathematically calculate the absolute maximum allowable Reaction Time ($T_R$) and Sensor Freshness Time ($T_F$) for a flight controller RTOS, guaranteeing flight stability.

**System Baseline (User Inputs):**

- **System:** 2D Quadcopter Pitch Axis
- **Moment of Inertia ($I$):** $0.015 \text{ kg} \cdot \text{m}^2$ need to determine
- **Motor Step Response ($63.2\%$ settling):** $50 \text{ ms}$ $(0.05 \text{ s})$ need to determine

## Step 1: Define the Macroscopic Plant (The Physics)

Before we can write software, we must mathematically define the physical limitations of the drone's hardware. The most critical limitation is the motor's responsiveness.

- **Motor Time Constant ($\tau_m$):** In physics, the time it takes a first-order system to reach $63.2\%$ of its final target value after a step command is its time constant.
  - Given your physical test: $\tau_m = 0.05 \text{ seconds}$.
- **Physical Bandwidth Limit:** A motor cannot physically change speeds faster than its time constant allows. The absolute maximum physical frequency the motor can output is calculated as the inverse of the time constant:
  - $\omega_{max} = \frac{1}{\tau_m}$
  - $\omega_{max} = \frac{1}{0.05} = \mathbf{20} \text{ rad/s}$ (approx $3.18 \text{ Hz}$).
  - *Meaning:* If the computer sends commands faster than $20 \text{ rad/s}$, the motor literally cannot spin up fast enough to obey. It becomes a physical bottleneck.

## Step 2: Establish the Crossover Frequency ($\omega_c$)

**What it is:** The Crossover Frequency ($\omega_c$) is the target "speed" of your PID controller. It dictates how aggressively the drone snaps to a commanded angle.

**Where the number comes from:** You cannot command the drone to move faster than its physical actuators. Therefore, your software's target crossover frequency *must* be lower than the motor's physical bandwidth limit ($\omega_{max}$).

- Our physical limit is $20 \text{ rad/s}$.
- To ensure the motor can faithfully follow the PID commands without saturating, control engineers typically set $\omega_c$ slightly below the physical pole.
- **Selected Target:** $\omega_c = 15 \text{ rad/s}$.

## Step 3: Define the Phase Margin and "Allowed Degradation"

**What Phase Margin (PM) is:** If a control loop experiences a $180^\circ$ delay between sensing and acting, it pushes when it should pull, causing violent, crashing oscillations. "Phase Margin" is your safety buffer—it is how many degrees of delay you can tolerate before hitting that deadly $180^\circ$ mark.

**Where the numbers come from:**

1. **The Ideal Target ($60^\circ$):** When tuning a PID controller mathematically (using standard loop shaping or Ziegler-Nichols frequency response methods), the mathematical ideal is a Phase

verify these numbers

Margin of $60^\circ$. This provides the perfect balance: the drone responds quickly but does not "bounce" (overshoot) when it stops.

2. **The Crash Limit ($45^\circ$):** In aerospace control theory, if the phase margin drops below $45^\circ$, the system becomes dangerously underdamped. The drone will shudder and ring violently. $45^\circ$ is the universally accepted minimum safety floor. verify this number
3. **The Allowed Degradation Budget ($\Delta\phi_{max}$):**
   - Ideal Mathematics = $60^\circ$ Phase Margin
   - Physical Crash Limit = $45^\circ$ Phase Margin
   - Difference = $15^\circ$
   - *Meaning:* The imperfections of reality (specifically, the delay caused by your digital computer) are allowed to eat exactly $15^\circ$ of your safety buffer. If the computer delays the signal by more than $15^\circ$, the drone crashes.

- **Convert to Radians for math:** $15^\circ \times (\frac{\pi}{180}) = \mathbf{0.2618}$ radians.
- This $0.2618$ rad is your absolute computational Phase Budget.

## Step 4: The Translation Equation (Digital Delay to Phase Loss)

Computers introduce delay in two ways. We must translate time delay (seconds) into phase loss (radians) based on how fast the drone is rotating ($\omega_c$).

**1. Reaction Time Delay ($T_R$):** This is the time it takes the CPU to read the IMU, run the PID math, and send the signal to the ESC.

- *Phase Loss:* If the drone is rotating at $\omega_c$ rad/s, and the CPU goes "blind" to do math for $T_R$ seconds, the drone rotates an unknown amount equal to Speed $\times$ Time.
- $\phi_R = \omega_c \cdot T_R$

**2. Sensor Freshness Delay ($T_F$ and the Zero-Order Hold):** The IMU takes a reading at time $0$, and the computer *holds* that exact number in memory until time $T_F$ (the next reading). This turns reality's smooth slope into a digital staircase.

- *Why exactly $T_F/2$?* Look at one step of a staircase. The flat line of the step crosses the true diagonal line of reality exactly in the middle of the step. Therefore, a digital staircase representation is mathematically identical to taking the true continuous reality and shifting it (delaying it) by exactly half the width of the step.
- $\phi_F = \omega_c \cdot \frac{T_F}{2}$

**The Final Constraint Equation:** Your computer's Phase Budget ($0.2618$ rad) must be greater than or equal to the CPU Math delay PLUS the Sensor Staircase delay.

$$0.2618 \geq \omega_c \cdot T_R + \omega_c \cdot \frac{T_F}{2}$$

## Step 5: Calculating the RTOS Deadlines

Now plug in our chosen $\omega_c$ ($15$ rad/s) to find the exact microsecond constraints for your RTOS scheduler.

$$0.2618 \geq 15 \cdot T_R + 15 \cdot \frac{T_F}{2}$$

As the firmware engineer, you allocate this budget based on your software architecture.

**Scenario A: Synchronous Pipeline** (The sensor is read at the exact same rate the PID loop runs: $T_R = T_F$)

- $0.2618 \geq 15(T_F) + 7.5(T_F)$
- $0.2618 \geq 22.5(T_F)$
- $T_F \leq 0.0116 \text{ seconds}$
- **Firmware Requirement:** You must configure your RTOS so that the IMU is polled and the PID loop executes at least every $11.6 \text{ milliseconds}$ ($\approx 86 \text{ Hz}$).

**Scenario B: Asynchronous Pipeline (Fast Sensor)** (You poll the IMU at a fixed high speed, e.g., $2 \text{ ms}$ Freshness, giving the CPU more time to run complex EKF/PID math).

- $0.2618 \geq 15(T_R) + 15(\frac{0.002}{2})$
- $0.2618 \geq 15(T_R) + 0.015$
- $0.2468 \geq 15(T_R)$
- $T_R \leq 0.0164 \text{ seconds}$
- **Firmware Requirement:** Because you reduced the sensor's staircase delay to $2 \text{ ms}$, the RTOS now has up to $16.4 \text{ milliseconds}$ to finish its heavy mathematical computations before sending the signal to the ESC.